1. Objetivo

Implementei o controle de embarque de passageiros em uma estação de metrô. Busquei respeitar a capacidade do carro e aguardar a conclusão de todos os embarques autorizados antes da partida. Entrego as funções de sincronização em C, a interface da estação, o programa de teste, o Makefile e o README, resultados obtidos em Linux.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementação em C executável em Linux	ok	Compilação e execução em evidencias.log
Estrutura da estação e quatro funções da interface	ok	sync_car.h, sync_car.c e vinculação do executável.
Uso das sete funções Pthreads exigidas	ok	Inicialização, mutex, espera, sinalização e broadcast em sync_car.c.
README explica arquivos, sistema e reproduçã	ok	Arquivo README
Makefile gera sync_car e oferece limpeza	ok	Compilação, limpeza e reconstrução, arquivo Makefile
Ausência de semáforos e de outras primitivas na solução	ok	Arquivos sync_car.c e sync_car.h
Apenas um mutex por estação	ok	Campo lock em struct station

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
Make run
```

4. Arquitetura

Arquivos e responsabilidades:

1. s ync_car.h: defini o tipo completo da estação e os protótipos utilizados pelo runner
2. s ync_car.c: implementei inicialização, reserva, confirmação e atendimento do carro.
3. sync_car-runner.c: utilizei o teste fornecido para criar passageiros e conferir as partidas

4. Makefile: Configurei compilação, dependência do cabeçalho, execução e limpeza.
5. evidencias.log: mantive o registro original do desenvolvimento

Decisões técnicas:

1. Um mutex por estação→preciso atualizar os contadores de forma consistente→reserva e confirmação alteram o estado dentro de seções críticas curtas

2. Duas variáveis de condição →passageiros e carro aguardam condições diferentes →car_available atende os passageiros e boarding_changed atende o carro

3. Reserva separada de confirmação →autorizar uma vaga não significa que o passageiro já se sentou →Mantenho boarding_passengers até a confirmação de cada reserva

4. Espera em laço→um despertar não garante que o estado ainda permite avançar→volto a verificar a condição depois de pthread_cond_wait

5. Estrutura definida no cabeçalho→o runner declara struct station station diretamente→ele precisa conhecer o tamanho e os campos do tipo durante a compilação.

5. Estratégias e Diário de Desenvolvimento

Em station_init, inicializo o mutex, as duas condições e os contadores. Não deve haver acesso concorrente durante essa inicialização.

Em station_wait_for_car, r egistro o passageiro na espera e bloqueio enquanto não existir embarque aberto com vaga disponível. Quando ele pode avançar, consumo uma vaga, retiro uma pessoa da espera e acrescento uma ao grupo em embarque, ainda sob o mutex. Depois libero o mutex e retorno.

Em station_on_board, diminui os embarques pendentes. Quando ninguém mais está embarcando e o carro está cheio ou não há passageiros esperando, sinalizo a condição do carro. A confirmação não devolve nem consome novamente a vaga.

Em station_load_car, anuncio as vagas, abro o embarque e acordo com os passageiros. Aguardo enquanto houver alguém embarcando ou houver vagas e pessoas esperando ao mesmo tempo. Antes de retornar, fecho o embarque e zero as vagas restantes.

Por exemplo, com cinco passageiros e três vagas, três reservas deixam dois esperando e três embarques pendentes. O carro só parte depois das três confirmações. Com dois passageiros e cinco vagas, ele pode partir com lugares vazios depois das duas confirmações, caso ninguém mais esteja esperando.

Verificar os retornos das operações Pthreads com check_pthread. Se uma delas falhar, mostro a descrição do código de erro e encerro o processo, em vez de continuar com recursos de sincronização inválidos.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Estado protegido e espera por condição	Separei espera, reserva e confirmação usando um mutex e duas condições. Desenvolvi as funções em etapas e comparei o resultado com o runner	Mantive a arquitetura, os ajustes ocorreram na implementação e nos testes

5.2 Diário de Tentativas

#	Estr.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Compilar os arquivos recebidos.	Falha de compilação.	Runner com instrução interrompida e definições repetidas.		Prints e evidencias.log
2	S1	Repetir make para conferir os erros.	Nova falha de compilação.	Os trechos inválidos do runner ainda estavam presentes.		Prints e evidencias.log
3	S1	Limpar e compilar com o runner recuperado.	Objetos gerados; vinculação falhou.	station_load_car e station_on_board ainda sem implementação.		Prints e evidencias.log
4	S1	Recompilar após implementar a espera dos passageiros.	Vinculação ainda falhou.	As funções do carro e de confirmação continuavam ausentes.		Prints e evidencias.log
5	S1	Executar make run.	Alvo sem regra.	O Makefile ainda não tinha a regra run.		Prints e evidencias.log
6	S1	Compilar em etapa posterior da implementação.	Executável gerado; aviso sobre foo.	—		Prints e evidencias.log
7	S1	Testar o embarque com make run.	Falha: carro retornou cedo.	Hipótese: espera insuficiente. O log não preserva o código exato dessa execução.		Prints e evidencias.log
8	S1	Limpar, recompilar e executar outra versão.	Looks good!; 100 passageiros atendidos.	—		Prints e evidencias.log
9	S1	Limpar e repetir a execução durante os experimentos.	Timeout após 2 s.	Hipóteses: sinalização insuficiente ou espera incorreta; causa não confirmada.		Prints e evidencias.log
1-	S1	Recompilar e executar a versão ao final da etapa.	Looks good!; 100 passageiros em 6 carros.	—		Prints e evidencias.log
11	S1	Confirmar embarque sem reservar vaga.	Limitação: contador igual a −1.	A chamada fora do contrato não é rejeitada; falta validação defensiva.		Prints e evidencias.log

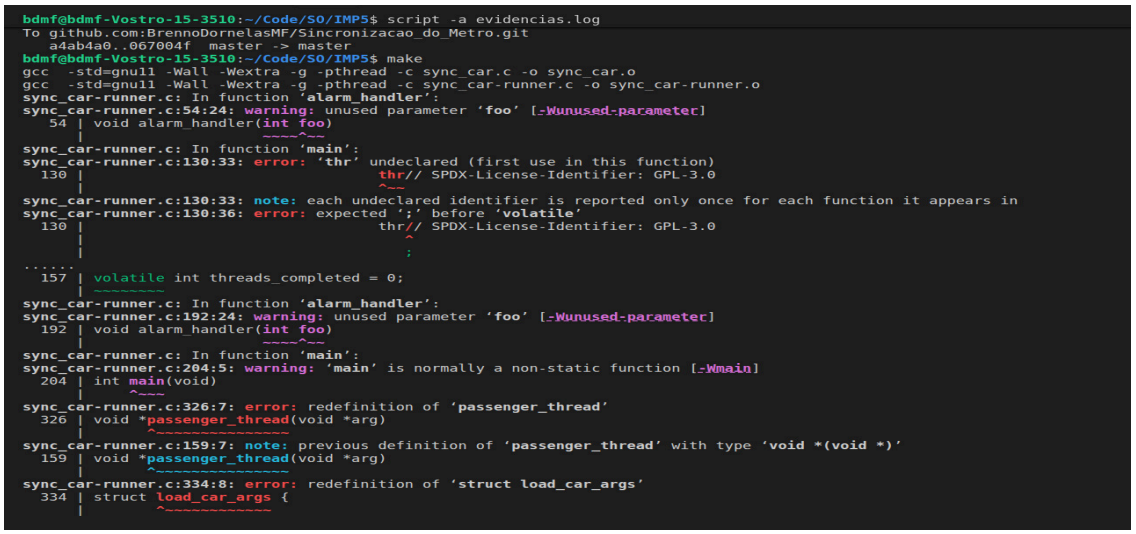
6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático

```
script -a evidencias.log
```

6.2 Prints

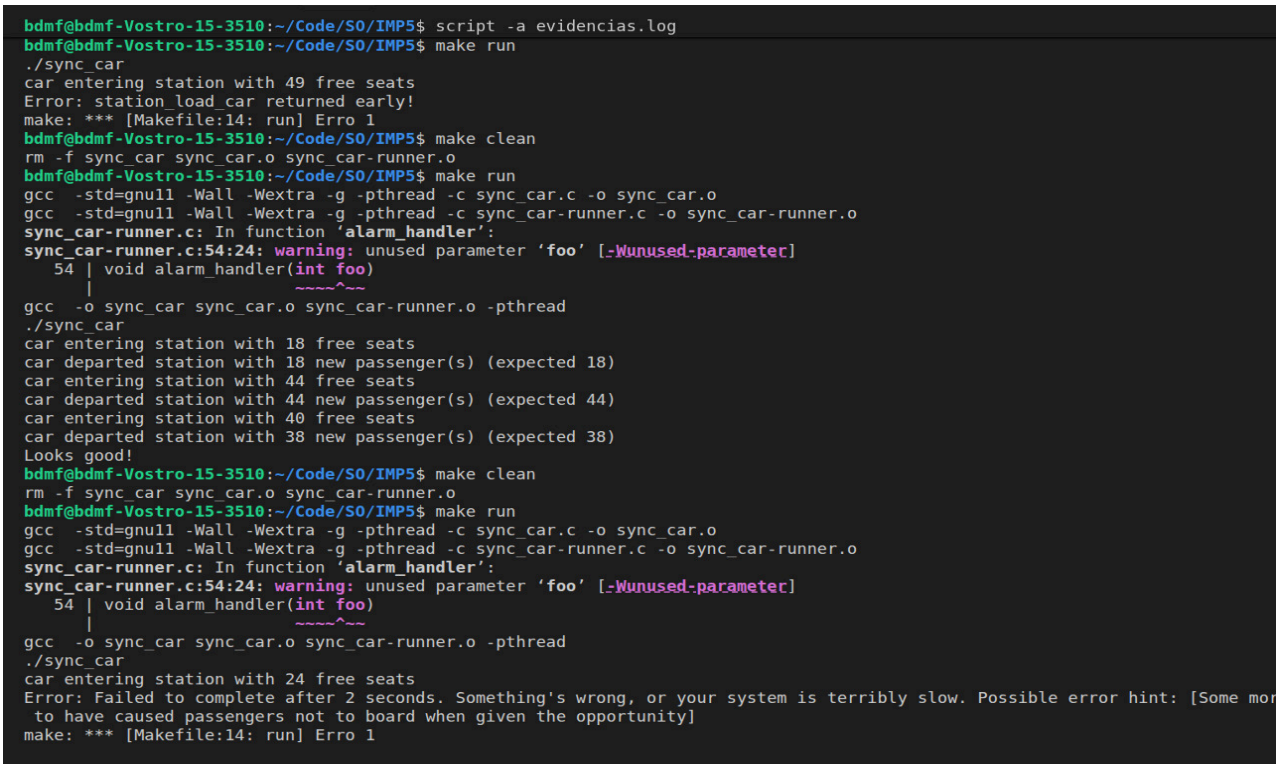

bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ script -a evidencias.log
To github.com:BrennoDornelasMF/Sincronizacao_do_Metro.git
a4ab4a0..067004f master -> master
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make
gcc -std=gnu11 -Wall -Wextra -g -pthread -c sync_car.c -o sync_car.o
gcc -std=gnu11 -Wall -Wextra -g -pthread -c sync_car-runner.c -o sync_car-runner.o
sync_car-runner.c: In function 'alarm_handler':
sync_car-runner.c:54:24: warning: unused parameter 'foo' [-Wunused-parameter]
54 | void alarm_handler(int foo)
| ~~~~^~~
sync_car-runner.c: In function 'main':
sync_car-runner.c:130:33: error: 'thr' undeclared (first use in this function)
130 | thr// SPDX-License-Identifier: GPL-3.0
| ^~~
sync_car-runner.c:130:33: note: each undeclared identifier is reported only once for each function it appears in
sync_car-runner.c:130:36: error: expected ';' before 'volatile'
130 | thr// SPDX-License-Identifier: GPL-3.0
| ^
| ;
.....
157 | volatile int threads_completed = 0;
| ~~~~~~~~
sync_car-runner.c: In function 'alarm_handler':
sync_car-runner.c:192:24: warning: unused parameter 'foo' [-Wunused-parameter]
192 | void alarm_handler(int foo)
| ~~~~^~~
sync_car-runner.c: In function 'main':
sync_car-runner.c:204:5: warning: 'main' is normally a non-static function [-Wmain]
204 | int main(void)
| ^~~~
sync_car-runner.c:326:7: error: redefinition of 'passenger_thread'
326 | void *passenger_thread(void *arg)
| ^~~~~~~~~~~~~~~~
sync_car-runner.c:159:7: note: previous definition of 'passenger_thread' with type 'void *(void *)'
159 | void *passenger_thread(void *arg)
| ^~~~~~~~~~~~~~~~
sync_car-runner.c:334:8: error: redefinition of 'struct load_car_args'
334 | struct load_car_args {
| ^~~~~~~~~~~~~

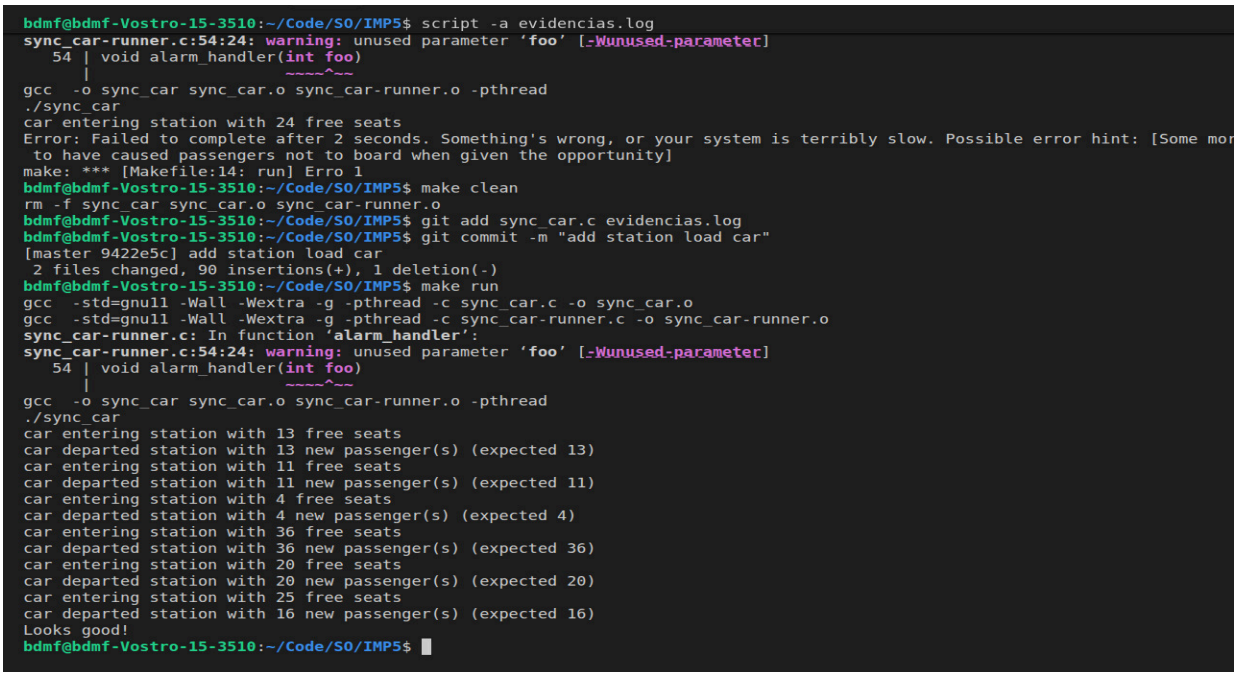
```
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ script -a evidencias.log
  183 | int alarm_timeout;
      |     ^~~~~~~~~~~~~
sync_car-runner.c:352:6: error: redefinition of ‘_alarm’
  352 | void _alarm(int seconds, const char *error_str)
      |      ^~~~~
sync_car-runner.c:185:6: note: previous definition of ‘_alarm’ with type ‘void(int,  const char *)’
  185 | void _alarm(int seconds, const char *error_str)
      |      ^~~~~
sync_car-runner.c:359:6: error: redefinition of ‘alarm_handler’
  359 | void alarm_handler(int foo)
      |      ^~~~~~~~~~~~~
sync_car-runner.c:192:6: note: previous definition of ‘alarm_handler’ with type ‘void(int)’
  192 | void alarm_handler(int foo)
      |      ^~~~~~~~~~~~~
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:359:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
  359 | void alarm_handler(int foo)
      |                    ~~~~^~
sync_car-runner.c: In function ‘main’:
sync_car-runner.c:371:5: warning: ‘main’ is normally a non-static function [-Wmain]
  371 | int main(void)
      |     ^~~~
sync_car-runner.c:371:5: error: redefinition of ‘main’
sync_car-runner.c:204:5: note: previous definition of ‘main’ with type ‘int(void)’
  204 | int main(void)
      |     ^~~~
sync_car-runner.c:482:2: error: ‘eads_reaped’ undeclared (first use in this function); did you mean ‘threads_reaped’?
  482 | }eads_reaped++;
      |  ^~~~~~~~~~
      |  threads_reaped
sync_car-runner.c: At top level:
sync_car-runner.c:371:5: warning: ‘main’ defined but not used [-Wunused-function]
  371 | int main(void)
      |     ^~~~
sync_car-runner.c:204:5: warning: ‘main’ defined but not used [-Wunused-function]
  204 | int main(void)
      |     ^~~~
make: *** [Makefile:17: sync_car-runner.o] Erro 1
```

```
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ script -a evidencias.log
  comando 'runc' do deb runc (1.3.4-0ubuntu1~24.04.1)
  comando 'rup' do deb rstat-client (4.0.1-11)
  comando 'crun' do deb crun (1.12-1)
Veja 'snap info <snapname>' para versões adicionais.
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ ./sync_car.o
bash: ./sync_car.o: Permissão negada
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make clena
make: *** Sem regra para processar o alvo 'clena'.  Pare.
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make clean
rm -f sync_car sync_car.o sync_car-runner.o
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ git add Makefile sync_car.c evidencias.log
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ git commit -m "add station on board"
[master f9f25ab] add station on board
 3 files changed, 90 insertions(+), 2 deletions(-)
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ git add README
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ git commit -m "add README"
[master 9cac832] add README
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.85 KiB | 632.00 KiB/s, done.
Total 8 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To github.com:BrennoDornelasMF/Sincronizacao_do_Metro.git
   29b1d4d..9cac832  master -> master
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car.c -o sync_car.o
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car-runner.c -o sync_car-runner.o
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~
gcc  -o sync_car sync_car.o sync_car-runner.o -pthread
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$
```

```
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ script -a evidencias.log
Comando 'run' não encontrado, você quis dizer:
  comando 'rue' do snap darkdimension-rue (1.0.7)
  comando 'runq' do deb exim4-daemon-heavy (4.97-4ubuntu4.6)
  comando 'runq' do deb exim4-daemon-light (4.97-4ubuntu4.6)
  comando 'runq' do deb sendmail-bin (8.17.2-1)
  comando 'zun' do deb python3-zunclient (4.7.0-0ubuntu1)
  comando 'rn' do deb trn4 (4.0-test77-16)
  comando 'grun' do deb grun (0.9.3+git20200303-3)
  comando 'zrun' do deb moreutils (0.67-1)
  comando 'rup' do deb rstat-client (4.0.1-11)
  comando 'srun' do deb slurm-client (23.02.6-1ubuntu2)
  comando 'crun' do deb crun (1.12-1)
  comando 'runc' do deb runc (1.3.4-0ubuntu1~24.04.1)
Veja 'snap info <snapname>' para versões adicionais.
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make run
./sync_car
car entering station with 49 free seats
Error: station_load_car returned early!
make: *** [Makefile:14: run] Erro 1
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make clean
rm -f sync_car sync_car.o sync_car-runner.o
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make run
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car.c -o sync_car.o
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car-runner.c -o sync_car-runner.o
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
gcc  -o sync_car sync_car.o sync_car-runner.o -pthread
./sync_car
car entering station with 18 free seats
car departed station with 18 new passenger(s) (expected 18)
car entering station with 44 free seats
car departed station with 44 new passenger(s) (expected 44)
car entering station with 40 free seats
car departed station with 38 new passenger(s) (expected 38)
Looks good!
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$
```

```
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ script -a evidencias.log
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.85 KiB | 632.00 KiB/s, done.
Total 8 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To github.com:BrennoDornelasMF/Sincronizacao_do_Metro.git
   29b1d4d..9cac832  master -> master
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car.c -o sync_car.o
gcc  -std=gnu11 -Wall -Wextra -g -pthread -c sync_car-runner.c -o sync_car-runner.o
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
gcc  -o sync_car sync_car.o sync_car-runner.o -pthread
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ run
Comando 'run' não encontrado, você quis dizer:
  comando 'rue' do snap darkdimension-rue (1.0.7)
  comando 'runq' do deb exim4-daemon-heavy (4.97-4ubuntu4.6)
  comando 'runq' do deb exim4-daemon-light (4.97-4ubuntu4.6)
  comando 'runq' do deb sendmail-bin (8.17.2-1)
  comando 'zun' do deb python3-zunclient (4.7.0-0ubuntu1)
  comando 'rn' do deb trn4 (4.0-test77-16)
  comando 'grun' do deb grun (0.9.3+git20200303-3)
  comando 'zrun' do deb moreutils (0.67-1)
  comando 'rup' do deb rstat-client (4.0.1-11)
  comando 'srun' do deb slurm-client (23.02.6-1ubuntu2)
  comando 'crun' do deb crun (1.12-1)
  comando 'runc' do deb runc (1.3.4-0ubuntu1~24.04.1)
Veja 'snap info <snapname>' para versões adicionais.
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$ make run
./sync_car
car entering station with 49 free seats
Error: station_load_car returned early!
make: *** [Makefile:14: run] Erro 1
bdmf@bdmf-Vostro-15-3510:~/Code/SO/IMP5$
```





7. Uso de IA

Onde usei IA :

Usei IA durante o desenvolvimento para entender as primitivas de sincronização, discutir exemplos de implementação e interpretar erros. Também obtive apoio na preparação do Makefile.

Prompts principais (

- Como proteger os contadores com um mutex?
- Como usar condições para bloquear passageiros e aguardar os embarques?
- Como revisar o Makefile, os erros e os cenários de teste?

8. Reflexão Final

Reflexão:

Separei reserva e confirmação para impedir a partida antes do fim dos embarques. O mutex protege os contadores, e as condições evitam espera ocupada na implementação. Aprendi a distinguir erros de compilação e vinculação de falhas de sincronização. O retorno antecipado e o timeout mostraram que passar uma vez não encerra a validação. Na revisão, 21 execuções passaram no runner; os cenários controlados ampliaram os testes. Na próxima, registraria cada versão testada e validaria chamadas inválidas desde o início. Com mais tempo, revisaria também a comunicação entre as threads do runner.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	ok
Rodei os testes e evidencias.log foi gerado	ok
Tenho prints obrigatórios	ok
Testei pelo menos um caso limite e um caso inválido	ok
Preenchi a seção de uso de IA	ok
Revisei o relatório e removi frases genéricas / vazias	ok

10. Se eu tivesse mais 2 horas

Definiria e testaria a rejeição de confirmações sem reserva e de capacidades negativas.Verificaria que essas entradas não alteram contadores nem deixam threads bloqueadas. Registraria o commit e o horário de cada experimento, ligando o resultado à versão testada. Testaria novas chegadas durante um embarque aberto, com confirmações controladas Revisaria a comunicação do runner e acrescentaria pthread_join ao encerrar as threads. Manteria -Wall -Wextra -Werror e repetiria os cenários após cada alteração. Acrescentaria a destruição do mutex e das condições ao fim do uso da estação.